DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOps

TP 9 — Conteneuriser un modèle avec Docker et le déployer sur Kubernetes

Durée : 3 h 30 · Partie 7 du cours · Windows PowerShell

Ce TP met en pratique la partie 7 du cours en deux temps. Partie A : vous prenez en main Docker, puis vous conteneurisez le service de détection de fraude en suivant les six étapes du cours (projet, service, dépendances, Dockerfile, construction, test). Partie B : vous déployez ce conteneur sur un orchestrateur, Kubernetes, dans un cluster local créé avec kind : plusieurs copies, une adresse unique, montée en charge, auto-réparation, mise à jour sans coupure et retour arrière après une version en panne. Le fil rouge reste le même : le dossier de Mme Benali doit donner une probabilité de 0,8899, sur votre machine, dans Docker et dans Kubernetes.

VERSION WINDOWS

Toutes les commandes sont écrites pour Windows PowerShell (5.1 ou 7), jamais l'invite cmd.exe. Les blocs `@' ... '@` écrivent un fichier ou l'envoient à Python : le marqueur de fermeture `'@` doit être en tout début de ligne, et le bloc se colle en entier. Les fichiers sont écrits en `-Encoding ascii` : l'encodage UTF8 de PowerShell 5.1 ajoute un marqueur invisible en tête de fichier qui casse la lecture des fichiers YAML et JSON.
Si l'activation de l'environnement est refusée : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`, puis relancez `.venv\Scripts\Activate.ps1`.

CE QUI CHANGE DANS CETTE VERSION

Le TP suit désormais le P07 étape par étape, avec les mêmes fichiers. Image de base Python 3.12. Le Dockerfile n'a plus de commentaires en fin de ligne : Docker ne les accepte pas (un « # » n'est un commentaire qu'en début de ligne). Nouveau : premières commandes Docker, page /docs, client de test dans le cluster, version en panne et retour arrière. Toutes les sorties ont été obtenues réellement.

Objectifs pédagogiques

- Utiliser les commandes de base de Docker : run, ps, logs, exec, stop, rm, images
- Conteneuriser un modèle de ML en six étapes et retrouver la même prédiction (0,8899)
- Mesurer l'effet de l'ordre des instructions du Dockerfile sur le temps de reconstruction
- Créer un cluster Kubernetes local et y déployer l'image avec un Deployment et un Service
- Observer l'évolutivité (plusieurs pods, répartition des requêtes) et l'auto-réparation
- Réaliser une mise à jour progressive sans erreur, puis un retour arrière après une version en panne

Prérequis

- Docker installé et démarré : Docker Desktop sous Windows (avec WSL 2) ou macOS, Docker Engine sous Linux. Prévoir 8 Go de mémoire et une connexion Internet pour télécharger les images
- Le projet `tp-mlops` des TP précédents, avec `models/model.pkl` (forêt du TP 3, entraînée avec scikit-learn 1.9.1)
- Le cours P07 sous les yeux : les six étapes et le mémo des commandes

Installation

Docker Desktop suffit pour la partie A. Pour la partie B, installez kubectl (le client Kubernetes ; Docker Desktop en fournit souvent déjà un) et kind (« Kubernetes in Docker »). Fermez puis rouvrez PowerShell après l'installation.

terminal

```
docker --version                    # Docker est-il installe ?
docker info | Select-Object -First 3  # le moteur repond-il ?
winget install -e --id Kubernetes.kubectl
winget install -e --id Kubernetes.kind
# fermer puis rouvrir PowerShell
kubectl version --client; kind version
```

RÉSULTAT ATTENDU

```
Docker version 29.x.x, build ...
Client Version: v1.3x.x
kind v0.33.0 go1.26.7 linux/amd64   (ou plus recent ; windows/amd64 sous Windows)
```

Déroulement

Étape 1 — Partie A · Premiers conteneurs (10 min)

On vérifie que Docker fonctionne et on observe ce qu'est un conteneur : un environnement isolé, avec son propre Python, lancé à partir d'une image.

terminal

```
python --version                                    # le Python de votre machine
docker run --rm python:3.12-slim python --version   # celui du conteneur
docker run --rm python:3.12-slim python -c "print('Bonjour depuis un conteneur')"
docker run --name essai python:3.12-slim python --version   # sans --rm
docker ps                                           # conteneurs en marche
docker ps -a                                        # tous les conteneurs
docker rm essai
docker images                                       # images presentes
```

RÉSULTAT ATTENDU

```
Python 3.1x.x               (votre machine : peu importe la version)
Unable to find image 'python:3.12-slim' locally   (premiere fois : telechargement)
Python 3.12.x
Bonjour depuis un conteneur
Python 3.12.x
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS    PORTS     NAMES     (vide)
CONTAINER ID   IMAGE              COMMAND              STATUS                                NAMES
49c8b0fb895b   python:3.12-slim   "python --version"   Exited (0) il y a 2 s   essai
essai
```

Si ça coince : « Cannot connect to the Docker daemon » : Docker n'est pas démarré. Lancez Docker Desktop (Windows, macOS) ou `sudo systemctl start docker` (Linux). Sous Linux, « permission denied » : ajoutez-vous au groupe docker (`sudo usermod -aG docker $USER`) puis reconnectez-vous.

Étape 2 — Organiser le projet (5 min)

Étape 1 du cours. Le service est rangé dans un sous-dossier `fraude-api` du projet `tp-mlops`, avec une copie du modèle du TP 3. Vérifiez au passage la version de scikit-learn qui a entraîné le modèle : c'est elle qu'il faudra installer dans l'image.

terminal

```
cd tp-mlops; .venv\Scripts\Activate.ps1
python -c "import sklearn; print(sklearn.__version__)"   # version d'entrainement
New-Item -ItemType Directory -Force fraude-api\api, fraude-api\models | Out-Null
Copy-Item models\model.pkl fraude-api\models
cd fraude-api
```

RÉSULTAT ATTENDU

```
1.9.1
```

À la fin de la partie A, le dossier contiendra : `api/main.py`, `api/client.py`, `models/model.pkl`, `requirements.txt`, `Dockerfile` et `.dockerignore`. Toutes les commandes suivantes se lancent depuis `fraude-api`.

Étape 3 — Écrire le service de prédiction (10 min)

Étape 2 du cours. Le modèle est chargé une seule fois ; `/health` indique que le service est prêt ; `/predict` renvoie la probabilité. Le champ `hote` renvoie le nom du conteneur qui a répondu : il servira à voir la répartition des requêtes en partie B. Le second fichier, `client.py`, est un petit client de test utilisé uniquement en partie B.

api/main.py

```
@'
import socket
import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel

VERSION = "1.0"                                    # version du service
HOTE = socket.gethostname()                        # nom du conteneur qui repond
modele = joblib.load("models/model.pkl")           # charge une seule fois, au demarrage

app = FastAPI(title="API de detection de fraude")


class Transaction(BaseModel):                      # les 4 entrees attendues
    montant: float
    anciennete: int
    freq_24h: int
    nuit: int


@app.get("/health")
def health():                                      # le service est-il pret ?
    return {"statut": "ok", "version": VERSION, "hote": HOTE}


@app.post("/predict")
def predict(t: Transaction):
    X = pd.DataFrame([t.model_dump()])
    p = float(modele.predict_proba(X)[0, 1])
    return {"fraude": p > 0.5, "probabilite": round(p, 4), "version": VERSION, "hote": HOTE}
'@ | Set-Content -Encoding ascii api/main.py
```

api/client.py (utilisé en partie B)

```
@'
"""Interroge le Service Kubernetes pendant N secondes et resume les reponses."""
import collections, json, sys, time, urllib.request

URL = "http://fraude-api/predict"            # le nom du Service suffit dans le cluster
DUREE = int(sys.argv[1]) if len(sys.argv) > 1 else 10
corps = json.dumps({"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}).encode()
par_pod, fin = collections.Counter(), time.time() + DUREE
while time.time() < fin:
    versions, pods, erreurs, t0 = collections.Counter(), set(), 0, time.time()
    while time.time() - t0 < 2:              # un bilan toutes les 2 secondes
        try:
            req = urllib.request.Request(URL, corps, {"Content-Type": "application/json"})
            r = json.load(urllib.request.urlopen(req, timeout=2))
            versions["v" + r["version"]] += 1
            pods.add(r["hote"])
            par_pod[r["hote"]] += 1
        except Exception:
            erreurs += 1
        time.sleep(0.05)
    bilan = dict(versions)
    print(time.strftime("%H:%M:%S"), bilan, "pods:", len(pods), "erreurs:", erreurs, flush=True)
print("reponses par pod :", dict(par_pod))
'@ | Set-Content -Encoding ascii api/client.py
```

Si ça coince : les fichiers sont volontairement écrits sans accents : sous Windows, l'écriture en `-Encoding ascii` remplacerait les lettres accentuées par des « ? ».

Étape 4 — Lister les dépendances (5 min)

Étape 3 du cours. Le signe `==` fige chaque version. scikit-learn doit être exactement la version affichée à l'étape 2 (1.9.1).

terminal

```
@'
fastapi==0.162.0
uvicorn==0.54.0
scikit-learn==1.9.1
pandas==3.1.0
joblib==1.6.0
'@ | Set-Content -Encoding ascii requirements.txt
```

Étape 5 — Écrire le Dockerfile et le .dockerignore (10 min)

Étape 4 du cours. Les commentaires sont sur leur propre ligne : dans un Dockerfile, un « # » placé en fin de ligne n'est pas un commentaire et provoque une erreur. Les bibliothèques sont installées avant la copie du code : c'est ce qui rend la reconstruction quasi instantanée (étape 9).

Dockerfile

```
@'
# 1. Image de depart : Python 3.12, version legere
FROM python:3.12-slim
# 2. Dossier de travail dans l'image
WORKDIR /app
# 3. Les bibliotheques d'abord (elles changent rarement)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# 4. Puis le code et le modele (ils changent souvent)
COPY api/ ./api/
COPY models/ ./models/
# 5. Le port du service et la commande de demarrage
EXPOSE 8000
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
'@ | Set-Content -Encoding ascii Dockerfile
```

.dockerignore

```
@'
.venv
data
__pycache__
.git
'@ | Set-Content -Encoding ascii .dockerignore
```

Si ça coince : le fichier s'appelle exactement `Dockerfile`, sans extension. Sous Windows, vérifiez avec `Get-ChildItem` qu'il ne s'appelle pas `Dockerfile.txt`.

Étape 6 — Construire l'image (10 min)

Étape 5 du cours. Le point final désigne le dossier courant. Notez la durée totale affichée sur la ligne « Building » : elle servira à l'étape 9.

terminal

```
docker build -t fraude-api:1.0 .
docker images fraude-api
```

RÉSULTAT ATTENDU

```
[+] Building 46.5s (11/11) FINISHED
 => [1/6] FROM docker.io/library/python:3.12-slim
 => [2/6] WORKDIR /app                                          0.4s
 => [3/6] COPY requirements.txt .                               0.0s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt  26.0s
 => [5/6] COPY api/ ./api/                                      0.1s
 => [6/6] COPY models/ ./models/                                0.0s
 => naming to docker.io/library/fraude-api:1.0

IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
fraude-api:1.0     fce813d29ffc        918MB          229MB

(durees et tailles variables selon la machine et la connexion)
```

Si ça coince : « failed to read dockerfile » : vous n'êtes pas dans le dossier `fraude-api`, ou le fichier porte une extension. Avec Docker Desktop, l'affichage de `docker images` peut être plus ancien (colonnes REPOSITORY, TAG, IMAGE ID, SIZE) : c'est la même information.

Étape 7 — Lancer et tester le conteneur (15 min)

Étape 6 du cours. `-d` lance en arrière-plan, `--name` donne un nom, `-p 8000:8000` relie le port 8000 de votre machine à celui du conteneur. Le service met environ 2 secondes à démarrer.

terminal

```
docker run -d --name fraude -p 8000:8000 fraude-api:1.0
docker ps
Invoke-RestMethod http://localhost:8000/health
$corps = '{"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}'
Invoke-RestMethod -Method Post -Uri http://localhost:8000/predict `
    -ContentType "application/json" -Body $corps
# requete incomplete (sans "nuit") : on affiche le code HTTP
try { Invoke-RestMethod -Method Post -Uri http://localhost:8000/predict `
        -ContentType "application/json" -Body '{"montant": 1800, "anciennete": 30, "freq_24h": 4}' }
catch { $_.Exception.Response.StatusCode.value__ }
```

RÉSULTAT ATTENDU

```
CONTAINER ID   IMAGE            COMMAND                  CREATED         STATUS        PORTS
          NAMES
3b7ebc7bd2d7   fraude-api:1.0   "uvicorn api.main:ap…"   2 seconds ago   Up 1 second   0.0.0.0:8000->80
00/tcp   fraude

statut version hote
------ ------- ----
ok     1.0     3b7ebc7bd2d7

fraude probabilite version hote
------ ----------- ------- ----
  True      0.8899 1.0     3b7ebc7bd2d7

422
```

Ouvrez ensuite http://localhost:8000/docs dans un navigateur : dépliez POST /predict, cliquez « Try it out », saisissez le dossier de Mme Benali puis « Execute ». La réponse doit contenir la même probabilité.

POINT DE CONTRÔLE

probabilite = 0.8899 pour Mme Benali, comme au TP 8. Une autre valeur indique un autre modèle ou une autre version de scikit-learn.

Si ça coince : sous PowerShell, `curl` est un alias de Invoke-WebRequest et les guillemets du JSON sont mal transmis à `curl.exe` : utilisez Invoke-RestMethod comme ci-dessus.

Étape 8 — Observer, puis arrêter et nettoyer (10 min)

terminal

```
docker logs fraude                                  # ce que le service ecrit
docker exec fraude python -c "import sklearn; print(sklearn.__version__)"
docker exec fraude ls /app /app/models              # le contenu du conteneur
docker stats --no-stream fraude                     # memoire et processeur
docker stop fraude
docker ps -a                                        # arrete, mais toujours la
docker rm fraude
docker ps -a                                        # supprime
```

RÉSULTAT ATTENDU

```
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 172.17.0.1:43306 - "GET /health HTTP/1.1" 200 OK
INFO: 172.17.0.1:43318 - "GET /health HTTP/1.1" 200 OK
INFO: 172.17.0.1:43334 - "POST /predict HTTP/1.1" 200 OK
INFO: 172.17.0.1:43346 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
1.9.1
/app:
api
models
requirements.txt

/app/models:
model.pkl
NAME      CPU %     MEM USAGE / LIMIT
fraude    0.16%     138.9MiB / 7.839GiB
fraude
... Exited (0) ...  fraude
fraude
(liste vide)
```

L'image `fraude-api:1.0`, elle, est toujours là : `docker run` la relance à l'identique en 2 secondes.

Étape 9 — Mesurer l'effet de l'ordre des instructions (15 min)

On modifie seulement le code, puis on reconstruit. Avec le Dockerfile du cours, l'installation des bibliothèques est reprise du cache. Avec un Dockerfile « mal ordonné », tout est réinstallé. Relevez les durées affichées sur la ligne « Building » des deux reconstructions.

terminal

```
Add-Content api\main.py "# modification"          # on modifie le code
docker build -t fraude-api:1.1 .                 # bon ordre
@'
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
'@ | Set-Content -Encoding ascii Dockerfile.mauvais
docker build -f Dockerfile.mauvais -t fraude-api:mauvais .   # premiere fois
Add-Content api\main.py "# modification 2"
docker build -f Dockerfile.mauvais -t fraude-api:mauvais .   # apres modification
```

RÉSULTAT ATTENDU

```
Bon ordre, apres modification du code :
 => CACHED [3/6] COPY requirements.txt .
 => CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt
 => [5/6] COPY api/ ./api/                        0.0s
[+] Building 0.6s

Mauvais ordre, apres modification du code :
 => [3/4] COPY . .                                0.6s
 => [4/4] RUN pip install --no-cache-dir -r requirements.txt   31.7s
[+] Building 50.3s
```

Supprimez ensuite les deux lignes « # modification » ajoutées à la fin de `api/main.py`, et le fichier `Dockerfile.mauvais`. Puis : `docker rmi fraude-api:1.1 fraude-api:mauvais`.

Étape 10 — Transporter l'image sans registre (10 min)

Portabilité : l'image devient un fichier que l'on peut copier sur une autre machine (clé USB, réseau), puis recharger. Comparez l'identifiant (colonne ID) avant et après.

terminal

```
docker images fraude-api:1.0                      # notez l'ID
docker save -o fraude-api-1.0.tar fraude-api:1.0
(Get-Item fraude-api-1.0.tar).Length / 1MB
docker rmi fraude-api:1.0
docker load -i fraude-api-1.0.tar
docker images fraude-api:1.0                      # meme ID ?
```

RÉSULTAT ATTENDU

```
-rw------- 219M fraude-api-1.0.tar
Untagged: fraude-api:1.0
Loaded image: fraude-api:1.0
IMAGE            ID             DISK USAGE   CONTENT SIZE   EXTRA
fraude-api:1.0   fce813d29ffc        918MB          229MB
```

Gardez ce fichier : c'est exactement ce que fait `kind load` à l'étape suivante pour transmettre l'image au cluster. Supprimez-le à la fin du TP.

Étape 11 — Partie B · Créer le cluster et lui donner l'image (10 min)

kind crée un cluster Kubernetes complet dans un conteneur Docker. Le cluster ne voit pas les images de votre machine : `kind load` les lui transmet. Sans cette commande, les pods restent bloqués en `ErrImagePull`.

terminal

```
kind create cluster --name tp-mlops               # 1 a 2 minutes la premiere fois
kubectl get nodes
kind load docker-image fraude-api:1.0 --name tp-mlops
```

RÉSULTAT ATTENDU

```
Creating cluster "tp-mlops" ...
 - Ensuring node image (kindest/node:v1.3x.x)
 - Preparing nodes
 - Starting control-plane
Set kubectl context to "kind-tp-mlops"

NAME                    STATUS   ROLES           AGE   VERSION
tp-mlops-control-plane   Ready    control-plane   40s   v1.3x.x

Image: "fraude-api:1.0" with ID "sha256:..." not yet present on node "tp-mlops-control-plane", loading.
..
```

Si ça coince : si `kubectl get nodes` affiche `NotReady`, attendez 30 secondes et relancez. Sous Windows, Docker Desktop doit être démarré avant `kind create cluster`.

Étape 12 — Décrire le déploiement : Deployment et Service (15 min)

Deux fichiers YAML décrivent l'état souhaité. Le Deployment demande 2 pods identiques à partir de l'image `fraude-api:1.0` et les surveille. Le Service donne à ces pods une adresse unique, `http://fraude-api`, et répartit les requêtes entre eux. Le lien entre les deux est l'étiquette `app: fraude-api`. Lisez chaque commentaire avant de lancer.

terminal

```
New-Item -ItemType Directory -Force k8s | Out-Null
@'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fraude-api
```

```
spec:
  replicas: 2                          # nombre de copies souhaitees
  selector:
    matchLabels:
      app: fraude-api                  # les pods geres : ceux qui portent cette etiquette
  template:                            # modele de chaque pod
    metadata:
      labels:
        app: fraude-api
    spec:
      containers:
        - name: api
          image: fraude-api:1.0        # l'image construite en partie A
          imagePullPolicy: IfNotPresent  # image locale, pas de telechargement
          ports:
            - containerPort: 8000
          lifecycle:
            preStop:                   # avant l'arret : 5 s pour finir les requetes en cours
              exec:
                command: ["sleep", "5"]
          readinessProbe:              # pas de trafic tant que /health ne repond pas
            httpGet:
              path: /health
              port: 8000
          resources:
            requests:                  # reserve pour chaque pod
              cpu: "100m"              # 0,1 coeur
              memory: "256Mi"
            limits:                    # plafond a ne pas depasser
              cpu: "500m"
              memory: "512Mi"
'@ | Set-Content -Encoding ascii k8s/deployment.yaml
```

terminal

```
@'
apiVersion: v1
kind: Service
metadata:
  name: fraude-api                     # nom utilise dans le cluster : http://fraude-api
spec:
  selector:
    app: fraude-api                    # envoie le trafic aux pods qui portent cette etiquette
  ports:
    - port: 80                         # port du Service
      targetPort: 8000                 # port du conteneur
'@ | Set-Content -Encoding ascii k8s/service.yaml
```

Ligne	Rôle
replicas: 2	nombre de pods identiques que Kubernetes maintient
selector / labels	relient le Deployment et le Service à leurs pods
imagePullPolicy	utiliser l'image chargée par kind, sans la télécharger
readinessProbe	aucune requête vers un pod tant que /health ne répond pas
preStop: sleep 5	avant l'arrêt d'un pod, 5 s pour finir les requêtes en cours
requests / limits	ressources réservées / plafond par pod

Si ça coince : YAML est sensible à l'indentation : uniquement des espaces, jamais de tabulations. Une erreur d'indentation donne « error converting YAML to JSON ».

Étape 13 — Déployer et observer (15 min)

terminal

```
kubectl apply -f k8s/
kubectl get pods -w                # Ctrl+C quand les 2 pods sont en 1/1
kubectl get deployments,services,pods
kubectl describe pod NOM_D_UN_POD   # copiez un nom affiche ci-dessus
kubectl logs NOM_D_UN_POD
```

RÉSULTAT ATTENDU

```
deployment.apps/fraude-api created
service/fraude-api created
$ kubectl get pods -w
NAME                           READY   STATUS              RESTARTS   AGE
fraude-api-7c4cc77476-5h4v9   0/1     ContainerCreating   0          0s
fraude-api-7c4cc77476-x6sj9   0/1     ContainerCreating   0          0s
fraude-api-7c4cc77476-x6sj9   0/1     Running             0          1s
fraude-api-7c4cc77476-5h4v9   0/1     Running             0          1s
fraude-api-7c4cc77476-x6sj9   1/1     Running             0          12s
fraude-api-7c4cc77476-5h4v9   1/1     Running             0          12s

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/fraude-api   2/2     2            2           20s

NAME                 TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
service/fraude-api   ClusterIP   10.43.7.60   <none>        80/TCP    20s
service/kubernetes   ClusterIP   10.43.0.1    <none>        443/TCP   76m

NAME                               READY   STATUS    RESTARTS   AGE
pod/fraude-api-7c4cc77476-5h4v9   1/1     Running   0          20s
pod/fraude-api-7c4cc77476-x6sj9   1/1     Running   0          20s

Events: (fin de kubectl describe)
  Normal   Scheduled  20s                default-scheduler  Successfully assigned default/fraude-api-7c
4cc77476-5h4v9 to tp-mlops-control-plane
  Normal   Pulled     20s                kubelet            Container image "fraude-api:1.0" already pr
esent on machine
  Normal   Created    20s                kubelet            Created container: api
  Normal   Started    20s                kubelet            Started container api
  Warning  Unhealthy  18s (x2 over 19s)  kubelet            Readiness probe failed: Get "http://10.42.0
.85:8000/health": dial tcp 10.42.0.85:8000: connect: connection refused

INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Les pods passent par `0/1 Running` (le conteneur démarre, la sonde /health n'a pas encore répondu) avant `1/1` (prêt). Les avertissements « Readiness probe failed » des premières secondes sont normaux.

Étape 14 — Appeler le service dans le cluster (10 min)

`kubectl port-forward` ouvre un tunnel entre votre machine et le Service. Il occupe le terminal : ouvrez un second terminal pour les requêtes.

terminal 1

```
kubectl port-forward service/fraude-api 8080:80
```

terminal 2

```
Invoke-RestMethod http://localhost:8080/health
$corps = '{"montant": 1800, "anciennete": 30, "freq_24h": 4, "nuit": 1}'
Invoke-RestMethod -Method Post -Uri http://localhost:8080/predict `
    -ContentType "application/json" -Body $corps
```

RÉSULTAT ATTENDU

```
Forwarding from 127.0.0.1:8080 -> 8000
Handling connection for 8080

{"statut":"ok","version":"1.0","hote":"fraude-api-7c4cc77476-5h4v9"}
{"fraude":true,"probabilite":0.8899,"version":"1.0","hote":"fraude-api-7c4cc77476-5h4v9"}
```

POINT DE CONTRÔLE

0.8899 dans Kubernetes aussi. Le champ hote contient maintenant le nom d'un pod.

Si ça coince : « lost connection to pod » dans le terminal 1 : le tunnel est lié à un seul pod ; s'il disparaît (étapes 16 à 18), relancez simplement `kubectl port-forward`.

Étape 15 — Vérifier la répartition des requêtes (10 min)

Le client de test (api/client.py) tourne dans un pod du cluster et appelle l'adresse du Service pendant 10 secondes. Il affiche toutes les 2 secondes les versions qui ont répondu, le nombre de pods différents et les erreurs, puis le total par pod.

terminal 2

```
kubectl run client --rm -i --restart=Never --image=fraude-api:1.0 -- python api/client.py 10
```

RÉSULTAT ATTENDU

```
19:07:25 {'v1.0': 26} pods: 2 erreurs: 0
19:07:27 {'v1.0': 26} pods: 2 erreurs: 0
19:07:29 {'v1.0': 27} pods: 2 erreurs: 0
19:07:31 {'v1.0': 28} pods: 2 erreurs: 0
19:07:33 {'v1.0': 27} pods: 2 erreurs: 0
reponses par pod : {'fraude-api-7c4cc77476-5h4v9': 67, 'fraude-api-7c4cc77476-x6sj9': 67}
pod "client" deleted from default namespace
```

Si ça coince : le message « If you don't see a command prompt, try pressing enter » peut s'afficher : ignorez-le. « pods "client" already exists » : `kubectl delete pod client`, puis relancez.

Étape 16 — Évolutivité : passer à 4 pods (10 min)

terminal 2

```
kubectl scale deployment fraude-api --replicas=4
kubectl get pods
kubectl get pods                    # quelques secondes plus tard
kubectl run client --rm -i --restart=Never --image=fraude-api:1.0 -- python api/client.py 6
```

RÉSULTAT ATTENDU

```
deployment.apps/fraude-api scaled
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
fraude-api-7c4cc77476-4wp6t   0/1     Running   0          3s
fraude-api-7c4cc77476-5h4v9   1/1     Running   0          36s
fraude-api-7c4cc77476-jd2f5   0/1     Running   0          3s
fraude-api-7c4cc77476-x6sj9   1/1     Running   0          36s
$ kubectl get pods   # quelques secondes plus tard
NAME                          READY   STATUS    RESTARTS   AGE
fraude-api-7c4cc77476-4wp6t   1/1     Running   0          13s
fraude-api-7c4cc77476-5h4v9   1/1     Running   0          46s
fraude-api-7c4cc77476-jd2f5   1/1     Running   0          13s
fraude-api-7c4cc77476-x6sj9   1/1     Running   0          46s

19:07:50 {'v1.0': 26} pods: 4 erreurs: 0
19:07:52 {'v1.0': 28} pods: 4 erreurs: 0
19:07:54 {'v1.0': 27} pods: 4 erreurs: 0
reponses par pod : {'fraude-api-7c4cc77476-5h4v9': 23, 'fraude-api-7c4cc77476-4wp6t': 24, 'fraude-api-7
c4cc77476-x6sj9': 18, 'fraude-api-7c4cc77476-jd2f5': 16}
pod "client" deleted from default namespace
```

Si ça coince : un pod reste en `Pending` ? Votre machine n'a plus assez de processeur ou de mémoire pour les `requests` demandés. `kubectl describe pod NOM` l'explique (« Insufficient cpu ») : revenez à 3 pods.

Étape 17 — Auto-réparation : supprimer un pod (10 min)

On simule une panne en supprimant un pod à la main. Kubernetes compare la réalité (3 pods) à l'état souhaité (4) et recrée aussitôt le pod manquant.

terminal 2

```
kubectl delete pod NOM_D_UN_POD
kubectl get pods
kubectl get pods                    # quelques secondes plus tard
kubectl scale deployment fraude-api --replicas=2   # on revient a 2 pods
```

RÉSULTAT ATTENDU

```
pod "fraude-api-7c4cc77476-4wp6t" deleted from default namespace
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
fraude-api-7c4cc77476-5f587   0/1     Running   0          6s
fraude-api-7c4cc77476-5h4v9   1/1     Running   0          61s
fraude-api-7c4cc77476-jd2f5   1/1     Running   0          28s
fraude-api-7c4cc77476-x6sj9   1/1     Running   0          61s
$ kubectl get pods   # quelques secondes plus tard
NAME                          READY   STATUS    RESTARTS   AGE
fraude-api-7c4cc77476-5f587   1/1     Running   0          12s
fraude-api-7c4cc77476-5h4v9   1/1     Running   0          67s
fraude-api-7c4cc77476-jd2f5   1/1     Running   0          34s
fraude-api-7c4cc77476-x6sj9   1/1     Running   0          67s
```

Repérez dans la colonne AGE le pod qui vient d'être créé. Personne ne l'a relancé à la main.

Étape 18 — Mise à jour progressive : version 2.0 sans coupure (20 min)

On publie une version 2.0 du service pendant que le client envoie des requêtes en continu. Kubernetes remplace les pods un par un : un nouveau pod ne reçoit du trafic qu'une fois prêt, et un ancien pod n'est arrêté qu'ensuite.

terminal 2 : construire et charger la version 2.0

```
(Get-Content api\main.py) -replace 'VERSION = "1.0"', 'VERSION = "2.0"' |
    Set-Content -Encoding ascii api\main.py
docker build -t fraude-api:2.0 .
kind load docker-image fraude-api:2.0 --name tp-mlops
```

terminal 1 : arrêtez le port-forward (Ctrl+C), puis lancez le client pendant 40 s

```
kubectl run client --rm -i --restart=Never --image=fraude-api:1.0 -- python api/client.py 40
```

terminal 2 : pendant que le client tourne

```
kubectl set image deployment/fraude-api api=fraude-api:2.0
kubectl rollout status deployment/fraude-api
kubectl rollout history deployment/fraude-api
```

RÉSULTAT ATTENDU

```
Terminal 2 :
deployment.apps/fraude-api image updated
$ kubectl rollout status deployment/fraude-api
Waiting for deployment "fraude-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "fraude-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "fraude-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "fraude-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "fraude-api" rollout to finish: 1 old replicas are pending termination...
deployment "fraude-api" successfully rolled out

deployment.apps/fraude-api
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

Terminal 1 (client) :
19:08:34 {'v1.0': 28} pods: 2 erreurs: 0
19:08:36 {'v1.0': 27} pods: 2 erreurs: 0
19:08:38 {'v1.0': 17, 'v2.0': 9} pods: 3 erreurs: 0
19:08:40 {'v2.0': 12, 'v1.0': 12} pods: 2 erreurs: 0
19:08:42 {'v1.0': 13, 'v2.0': 13} pods: 2 erreurs: 0
19:08:44 {'v2.0': 14, 'v1.0': 12} pods: 2 erreurs: 0
19:08:46 {'v1.0': 13, 'v2.0': 14} pods: 2 erreurs: 0
19:08:48 {'v1.0': 17, 'v2.0': 10} pods: 2 erreurs: 0
19:08:50 {'v2.0': 22, 'v1.0': 5} pods: 3 erreurs: 0
19:08:52 {'v2.0': 26} pods: 2 erreurs: 0
19:08:54 {'v2.0': 28} pods: 2 erreurs: 0
...
reponses par pod : {'fraude-api-7c4cc77476-x6sj9': 122, 'fraude-api-7c4cc77476-5h4v9': 186, 'fraude-api
-6f8c4cbc9-zxkgp': 153, 'fraude-api-6f8c4cbc9-mdglc': 77}
```

CE QU'IL FAUT VOIR

Pendant une douzaine de secondes, v1.0 et v2.0 répondent ensemble ; ensuite seule v2.0 répond. Mesure de préparation : 538 requêtes pendant la mise à jour, 0 erreur. Sans la ligne `preStop` du Deployment, nous avons mesuré 7 erreurs sur 533 requêtes.

Étape 19 — Une version en panne, puis retour arrière (20 min)

On publie volontairement une version 2.1 défectueuse : le chemin du modèle est faux, le service plante au démarrage. Observez que les clients ne voient rien, puis revenez à la version précédente.

terminal 2 : fabriquer la version en panne

```
(Get-Content api\main.py) -replace 'VERSION = "2.0"', 'VERSION = "2.1"' `
    -replace 'model.pkl', 'modele.pkl' |
    Set-Content -Encoding ascii api\main.py
docker build -t fraude-api:2.1 .
kind load docker-image fraude-api:2.1 --name tp-mlops
```

terminal 1 : le client, pendant 50 s

```
kubectl run client --rm -i --restart=Never --image=fraude-api:1.0 -- python api/client.py 50
```

terminal 2 : pendant que le client tourne

```
kubectl set image deployment/fraude-api api=fraude-api:2.1
kubectl rollout status deployment/fraude-api --timeout=40s
kubectl get pods
kubectl logs NOM_DU_POD_EN_ERREUR | Select-Object -Last 4
kubectl rollout undo deployment/fraude-api       # retour a la version precedente
kubectl rollout status deployment/fraude-api
kubectl get pods
kubectl rollout history deployment/fraude-api
```

RÉSULTAT ATTENDU

```
deployment.apps/fraude-api image updated
$ kubectl rollout status deployment/fraude-api --timeout=40s
Waiting for deployment "fraude-api" rollout to finish: 1 out of 2 new replicas have been updated...
error: timed out waiting for the condition
$ kubectl get pods
NAME                          READY   STATUS             RESTARTS      AGE
fraude-api-64d86b8f54-zjxhw   0/1     CrashLoopBackOff   2 (21s ago)   40s
fraude-api-6f8c4cbc9-mdglc    1/1     Running            0             71s
fraude-api-6f8c4cbc9-zxkgp    1/1     Running            0             83s

  File "/usr/local/python/lib/python3.12/site-packages/joblib/numpy_pickle.py", line 741, in load
    with open(filename, "rb") as f:
         ^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'models/modele.pkl'

deployment.apps/fraude-api rolled back
$ kubectl rollout status deployment/fraude-api
deployment "fraude-api" successfully rolled out

NAME                          READY   STATUS    RESTARTS   AGE
fraude-api-6f8c4cbc9-mdglc    1/1     Running   0          2m23s
fraude-api-6f8c4cbc9-zxkgp    1/1     Running   0          2m35s

deployment.apps/fraude-api
REVISION  CHANGE-CAUSE
1         <none>
3         <none>
4         <none>

Client (terminal 1) : uniquement {'v2.0': ...} et « erreurs: 0 » du debut a la fin
```

CE QU'IL FAUT VOIR

Le nouveau pod est en `CrashLoopBackOff` : il plante, Kubernetes le relance, il replante. Comme il n'est jamais prêt (readinessProbe), il ne reçoit aucune requête et les deux pods 2.0 continuent de servir : 673 requêtes, 0 erreur pendant la préparation. `rollout undo` revient à la 2.0 (révision 4 = ancienne révision 2).

Remettez le bon code : `VERSION = "1.0"` et `models/model.pkl` dans `api/main.py` (ouvrez le fichier et corrigez les deux lignes).

Étape 20 — Nettoyer (5 min)

terminal

```
kubectl delete -f k8s/
kind delete cluster --name tp-mlops
docker rmi fraude-api:2.0 fraude-api:2.1
Remove-Item fraude-api-1.0.tar
```

RÉSULTAT ATTENDU

```
deployment.apps "fraude-api" deleted from default namespace
service "fraude-api" deleted from default namespace
Deleting cluster "tp-mlops" ...
```

Gardez l'image `fraude-api:1.0` et le dossier `fraude-api` : ils serviront au TP 10 (API REST) et au TP 12 (déploiement dans le cloud).

Questions du compte rendu

1. Expliquez la différence entre une image et un conteneur à partir de l'étape 1. Après une modification de `api/main.py`, que faut-il faire pour que le conteneur utilise le nouveau code ?
2. Pourquoi `requirements.txt` utilise-t-il `==` ? Que risquerait-on avec `scikit-learn` sans version, sachant que le modèle a été entraîné avec la 1.9.1 ?
3. Donnez vos deux durées de reconstruction de l'étape 9 et expliquez l'écart avec les notions de couche et de cache.

4. Quelle est la différence entre `RUN` et `CMD` ? Pourquoi `--host 0.0.0.0` ? `EXPOSE 8000` suffit-il à rendre le service joignable depuis votre machine ?
5. Quel est le rôle du Deployment, du Service et de l'étiquette `app: fraude-api` ? Que se passerait-il si le `selector` du Service indiquait `app: fraude` ?
6. Pourquoi les pods passent-ils par `0/1` avant `1/1` ? Sans `readinessProbe`, que se serait-il passé pour les clients lors du déploiement de la version 2.1 ?
7. Combien d'erreurs le client a-t-il comptées pendant la mise à jour de l'étape 18 ? À quoi sert la ligne `preStop` ? (Facultatif : retirez-la, refaites une mise à jour et comparez.)
8. Remplissez le tableau « avantage / mesure obtenue » pour la cohérence (0,8899 dans les trois environnements), la portabilité (étape 10) et l'évolutivité (étapes 15 à 17).

Livrables

- Le dossier `fraude-api` : `api/`, `requirements.txt`, `Dockerfile`, `.dockerignore`, `k8s/` (sans le fichier .tar), commité dans Git
- Captures : réponse 0,8899 dans Docker (étape 7) et dans Kubernetes (étape 14)
- Les deux durées de reconstruction de l'étape 9
- Captures de `kubectl get pods` avant et après la suppression d'un pod (étape 17)
- La sortie du client pendant la mise à jour (étape 18) et pendant la version en panne (étape 19)
- Le compte rendu avec les réponses aux questions et le tableau de l'avantage / mesure

Barème

Critère	Points
Premiers conteneurs, projet, service, requirements.txt, Dockerfile (étapes 1 à 5)	3
Image construite, conteneur testé, 0,8899 retrouvé, observation et nettoyage (étapes 6 à 8)	3
Cache mesuré et expliqué, image transportée (étapes 9 et 10)	2
Cluster créé, Deployment et Service appliqués, service appelé : 0,8899 (étapes 11 à 14)	4
Répartition, montée à 4 pods, auto-réparation (étapes 15 à 17)	3
Mise à jour sans erreur, version en panne et retour arrière (étapes 18 et 19)	3
Réponses aux questions et tableau des avantages	2
Total	20